
Specification-Driven Synthetic Evaluation for Generative-Media MCP Tool Calling: Architectural Deconstruction, Rubric Mechanics, and Empirical Validation

G. Hussain Chinoy

ghchinoy@gmail.com

Independent Researcher

ABSTRACT

Evaluating autonomous AI agents interfacing with complex Model Context Protocol (MCP) tool suites presents a severe cold-start dilemma: manual scenario curation is labor-prohibitive, static benchmarks suffer rapid obsolescence, and live multi-turn execution is economically unviable in stochastic, high-latency generative-media domains. This work deconstructs and empirically validates the specification-driven evaluation methodology of Agent Seer (arXiv:2608.26133) across three production-grade generative-media MCP suites: Google Veo, Gemini Image/Nanobanana, and Google Lyria. We formalize a four-stage pipeline that ingests raw schemas to synthesize multi-turn enterprise scenarios with held-out oracle workflows, utilizing a decomposed LLM-as-judge rubric constrained by non-linear cascading penalties to prevent score dilution. Through systematic validation, we expose a critical vulnerability—Schema-Blindness—wherein standard JSON schemas omit runtime backend compatibility constraints, causing un-enriched judges to award perfect scores to production-breaking bugs. We demonstrate that injecting a machine-readable capability matrix restores robust discrimination gaps (≥ 0.191) across all suites and expands the image generation discrimination margin by +36.1%. Finally, we establish a three-layer generative media evaluation taxonomy that decouples low-cost orchestration correctness from expensive perceptual quality assessment, providing a robust paradigm for continuous integration gating.

1 1. Executive Summary & Problem Formulation

1.1 1.1 The Cold-Start Evaluation Problem in Agent Tool Calling

The rapid expansion of autonomous agent frameworks has transformed tool-calling from a simple single-turn API dispatch mechanism into multi-turn, multi-tool orchestration workflows. When agents interact with standardized interfaces such as Anthropic’s Model Context Protocol (MCP), validating tool-use competence becomes a gating requirement for production deployment. However, evaluating emerging, private, or rapidly iterating MCP servers encounters three fundamental bottlenecks:

1. **Curation Bottleneck:** Handcrafting multi-turn user prompts, gold-standard tool calls, and execution mocks requires prohibitive human engineering effort.
2. **Static Benchmark Rot:** Hardcoded evaluation suites rot as API schemas, parameter names, and model enums iterate across releases.
3. **Multi-Turn Evaluation Gap:** Downstream conversational turns require dynamic, realistic tool responses that reflect intermediate state without expensive execution.

In generative-media domains (video generation, image synthesis, audio composition), these challenges are magnified. Media diffusion backends involve long execution latencies (30–120 seconds per video clip), high cloud inference costs, and stochastic output variance. Running live execution loops simply to test whether an agent can correctly structure a tool call is economically and architecturally unviable.

1.2 1.2 Agent Seer Formulation & Core Thesis

Agent Seer (Karumuri et al., arXiv:2608.26133) establishes that **tool specifications**—consisting of **function signatures**, **natural-language documentation**, and **typed JSON schemas**—**encode sufficient semantic structure to autonomously synthesize end-to-end evaluation suites without manual authoring or live API execution.**

By feeding raw MCP schemas through a disciplined four-stage pipeline, the system extracts semantic affordances, generates diverse single-turn and multi-turn enterprise scenarios with held-out oracle workflows, synthesizes realistic mock responses across explicit grounding tiers, and evaluates agent transcripts using an unsupervised, decomposed LLM-as-judge rubric.

1.3 1.3 Scope of this Technical Report

This report presents an exhaustive engineering deconstruction and empirical reproduction of Agent Seer applied to three generative-media MCP server suites:

1. **Google Veo Server (mcp-veo-go):** 6 video generation and editing tools (veo_t2v, veo_i2v, veo_first_last_to_video, veo_reference_to_video, veo_ingredients_to_video, veo_extend_video).
2. **Gemini Image / Nanobanana Server (mcp-nanobanana-go):** Multi-model image generation tool (nanobanana_image_generation) supporting Gemini 2.5 Flash, Gemini 3 Pro, Gemini 3.1 Flash, and Gemini 3.1 Flash Lite.
3. **Google Lyria Server (mcp-lyria-go):** Audio and music generation tool (lyria_generate_music) supporting Lyria 2, Lyria 3 Clip, and Lyria 3 Pro.

2 2. 4-Stage Spec-Driven Pipeline Deconstruction

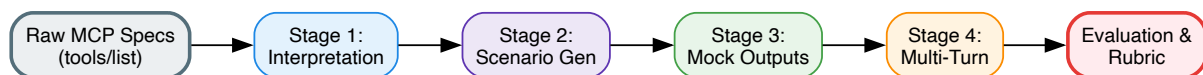


Figure 1: The Four-Stage Agent Seer Specification-Driven Evaluation Pipeline. Converts raw JSON-RPC schemas into complete evaluation trajectories using structured, unsupervised LLM tasks.

The synthetic generation pipeline converts raw schema definitions into validated benchmark harnesses through four sequential transformations, each constrained by formal input/output schemas.

2.1 2.1 Stage 1: Tool Interpretation (Semantic Feature Extraction)

Raw tool definitions provided by MCP `tools/list` are terse and optimized for parser consumption rather than conceptual reasoning. Stage 1 expands each raw tool schema into a rich 5-dimensional semantic representation:

1. `tool_name`: Exact string identifier of the MCP tool.
2. `what_it_does`: Exhaustive functional summary of capabilities, operational modalities, and transformation mechanics.
3. `what_it_needs`: Deconstructed inventory of mandatory versus optional parameters, accepted primitive and composite types, format constraints, and domain-specific valid ranges.
4. `why_its_used`: Strategic intent, task affordances, and execution rationales that distinguish this tool from adjacent APIs.
5. `enterprise_context`: High-level business and operational classification tags (e.g., ["Digital Asset Creation", "Marketing Automation", "Broadcast Post-Production"]).

2.1.1 Formal JSON Contract for Stage 1:

```
{
  "tool_name": "veo_first_last_to_video",
  "what_it_does": "Generates a smooth video transition connecting an initial start-frame image and a final end-frame image based on a guiding textual prompt.",
  "what_it_needs": {
    "required": ["first_frame_uri", "last_frame_uri", "prompt"],
    "optional": ["model", "aspect_ratio", "duration_seconds", "bucket"],
    "type_constraints": {
      "first_frame_uri": "string (valid gs:// Cloud Storage URI)",
      "last_frame_uri": "string (valid gs:// Cloud Storage URI)",
      "duration_seconds": "integer (model-constrained bounds)"
    }
  },
  "why_its_used": "Used when strict visual continuity is required between two predefined keyframes, preventing diffusion drift across scene boundaries.",
  "enterprise_context": ["Video Production", "VFX Previsualization", "Commercial Stitching"]
}
```

2.2 2.2 Stage 2: Scenario Generation (Simple vs. Complex & Oracle Workflows)

Stage 2 ingests the aggregated Stage 1 semantic summaries and synthesizes realistic task scenarios across two orthogonal complexity tiers:

- **Simple Tier (STAGE2_SIMPLE)**: Focuses on direct, single-intent user prompts requiring one or two deterministic tool calls with minimal branching.
- **Complex Tier (STAGE2_COMPLEX)**: Focuses on multi-faceted enterprise workflows requiring composite tool chaining, multimodal asset transformation, conditional parameter tuning, and multi-domain coordination.

2.2.1 The 100% Tool Coverage Suffix Guarantee

To eliminate generator selection bias (where the LLM repeatedly generates scenarios for prominent tools like `t2v` while ignoring niche tools like `veo_first_last_to_video`), the prompt injects a strict coverage constraint:

$$\forall t \in \mathcal{T}, \quad \exists s \in \mathcal{S} \quad \text{such that} \quad t \in \text{Workflow}(s) \quad (1)$$

Where $\mathcal{T}$ is the set of N available tools in the MCP suite and $\mathcal{S}$ is the set of generated scenarios. In our empirical reproduction on `mcp-veo-go` ($N = 6$), Stage 2 yielded **15 scenarios (6 simple, 9 complex)** with **100% tool coverage (0 uncovered tools)** across four enterprise categories: Creative Advertising, VFX Previsualization, Social Media Campaigning, and Multi-Asset Video Stitching.

2.2.2 Held-Out Oracle Workflow Structure

Every generated scenario produces an `agent_workflow` containing the exact sequence of expected function calls, fully specified parameter arguments, and step-level rationales:

```
{
  "title": "Seamless Interpolation for Automotive Commercial Transition",
  "novelty_reason": "Requires temporal keyframe interpolation between studio shot and road test shot with strict GCS URI validation.",
  "prompt": "We have a hero studio render at gs://auto-assets/studio_front.png and a desert highway shot at gs://auto-assets/highway_rear.png. Create a 5-second cinematic transition connecting them with dust kicking up.",
  "agent_workflow": [
    {
      "function_name": "veo_first_last_to_video",
      "parameters": {
        "first_frame_uri": "gs://auto-assets/studio_front.png",
        "last_frame_uri": "gs://auto-assets/highway_rear.png",
        "prompt": "Cinematic transition from studio render to desert highway, kicking up dust, dynamic camera glide",
        "duration_seconds": 5,
        "model": "veo-3.1-generate-preview"
      },
      "quick_explanation": "Interpolates between the two keyframe URIs using the preview model supporting first/last conditioning."
    }
  ]
}
```

2.3 Stage 3: Mock Output Generation with Grounding Tiers

Stage 3 resolves the downstream multi-turn dependency problem without executing live tools. For every step in the `agent_workflow`, synthetic tool execution responses are generated. To quantify the fidelity of these mocks, Agent Seer introduces an explicit **Grounding Tier Taxonomy**:

Grounding Tier	Source Definition	Semantic Confidence Tag
High Grounding	Grounded in verified runtime execution schema or real success response for the exact tool.	"confidence": "high"
Medium Grounding	Grounded in analogous tool outputs within the same server family or related schema shape.	"confidence": "medium"

Grounding Tier	Source Definition	Semantic Confidence Tag
Low Grounding	Spec-only synthesis without runtime response examples (pure LLM hallucination).	"confidence": "low"

2.3.1 Empirical Grounding Distribution Analysis

In the original paper, 100% of generated mock outputs operated at `low` grounding due to the absence of reference responses in open-source specifications.

In our reproduction, we seeded Stage 3 with real response fixtures (`spike/seed_outputs.json`) derived from `video_logic.go` and verified via `smoke_generate_and_verify.sh`. Across all 15 scenarios comprising **19 individual workflow steps**, our empirical distribution achieved:

- **High Grounding: 16 / 19 steps (84.2%)** — Emitted fully compliant response payloads combining structured status text and standard `resource_link` metadata blocks.
- **Medium Grounding: 3 / 19 steps (15.8%)** — Occurred exclusively in Scenario 13 (“Rapid Prototyping of Marketing Concepts”), where 3 sequential `veo_i2v` iterations synthesized distinct variation IDs.
- **Low Grounding: 0 / 19 steps (0.0%)**.

```
// Representative High-Grounding Mock Output (Veo Server)
{
  "function_name": "veo_first_last_to_video",
  "confidence": "high",
  "mock_output": {
    "status": "completed",
    "operation_id": "projects/108284920/locations/us-central1/publishers/google/
models/veo-3.1/operations/interp_8921b",
    "response": {
      "videos": [
        {
          "gcs_uri": "gs://auto-assets/veo_output/transition_interp_8921b.mp4",
          "mime_type": "video/mp4",
          "duration_seconds": 5
        }
      ]
    }
  },
  "text": "Successfully rendered 5s first-to-last keyframe transition video."
}
```

2.4 2.4 Stage 4: Multi-Turn Expansion (Phase Boundaries & State Chaining)

Stage 4 segments composite workflows at natural phase boundaries to evaluate multi-step and multi-hop agent capabilities (aligned with Berkeley Function Calling Benchmark / BFCL v3 patterns):

1. **Turn 1 (Initial Phase):** User issues the overarching objective. The agent executes initial generation or asset preparation calls. The evaluation harness captures emitted calls, scores them, and injects the corresponding Stage 3 mock outputs.

2. **Turn $t + 1$ (Follow-Up Phase):** The user or agent initiates follow-up operations referencing state dynamically created in Turn t (e.g., extending a video using the output URI from Turn 1, or applying audio dubbing to an emitted video stem).
3. **State Consistency Verification:** The judge evaluates whether the agent properly extracts dynamic identifiers (`operation_id`, `gcs_uri`) emitted in prior mock turns or hallucinates nonexistent paths.

3.3. LLM-as-Judge Decomposed Rubric & Scoring Mechanics

The evaluation engine separates assessment into two orthogonal domains: **Tool-Calling Correctness (TC)** and **Conversational Coherence (Coh)**.

3.1.1 Mathematical Formulation of Tool-Calling Correctness (TC)

Every sub-dimension k is scored by the judge on a discrete integer scale $x_k \in \{0, 1, \dots, 10\}$. Scores are normalized to the unit interval $[0.0, 1.0]$ via:

$$\text{norm}_{10}(x_k) = \max\left(0.0, \min\left(1.0, \frac{x_k}{10.0}\right)\right) \quad (2)$$

3.1.1.1 The 14 Sub-Dimensions Across 4 Core Categories:

Idx	Category	Subdimension (k)	Scope	Normalization	Evaluation Focus
1	Usage	necessity	Always	$D_{\text{usage}} = \text{norm}_{10}(x_{\text{nec}})$	Was a tool call required, or could the LLM answer directly?
2		overuse_detected	Diagnostic	Excluded from aggregate	Did the agent make redundant or unprompted calls?
3	Selection	correctness	Always	$\text{norm}_{10}(x_{\text{cor}})$	Does the tool choice match the requested functional intent?
4		specificity	Always	$\text{norm}_{10}(x_{\text{spec}})$	Was the most specialized tool selected over generic tools?
5		completeness	Always	$\text{norm}_{10}(x_{\text{comp}})$	Were all necessary tools selected to satisfy the task?
6	Arguments	completeness	Always	$\text{norm}_{10}(x_{\text{arg_comp}})$	Are all mandatory schema parameters provided?
7		name_accuracy	Always	$\text{norm}_{10}(x_{\text{name}})$	Do parameter keys match the schema exactly (case-sensitive)?

Idx	Category	Subdimension (k)	Scope	Normalization	Evaluation Focus
8		value_accuracy	Always	$\text{norm}_{10}(x_{\text{val}})$	Are values grounded, valid, and aligned with prompt/context?
9		type_compliance	Always	$\text{norm}_{10}(x_{\text{type}})$	Do values match expected types (string, int, array, object)?
10		format_compliance	Always	$\text{norm}_{10}(x_{\text{fmt}})$	Do values follow formats (URI schemes, enums, bounds)?
11		relevancy	Always	$\text{norm}_{10}(x_{\text{rel}})$	Are arguments free of ungrounded or extraneous keys?
12	Ordering	sequence_logic	Tools > 1	$\text{norm}_{10}(x_{\text{seq}})$	Is execution order logical across dependent steps?
13		dependency_handling	Tools > 1	$\text{norm}_{10}(x_{\text{dep}})$	Are output values from earlier steps piped correctly?
14		execution_efficiency	Tools > 1	$\text{norm}_{10}(x_{\text{eff}})$	Is the execution path optimal without redundant hops?

3.1.2 Dimension and Overall Score Aggregation Formulas:

1. Usage Dimension (D_{usage}):

$$D_{\text{usage}} = \text{norm}_{10}(x_{\text{necessity}}) \quad (3)$$

2. Selection Dimension ($D_{\text{selection}}$):

$$D_{\text{selection}} = \frac{1}{3}(\text{norm}_{10}(x_{\text{cor}}) + \text{norm}_{10}(x_{\text{spec}}) + \text{norm}_{10}(x_{\text{comp}})) \quad (4)$$

3. Arguments Dimension ($D_{\text{arguments}}$):

$$D_{\text{arguments}} = \frac{1}{6} \sum_{k \in \mathcal{K}_{\text{arg}}} \text{norm}_{10}(x_k) \quad (5)$$

where $\mathcal{K}_{\text{arg}} = \{\text{completeness, name_accuracy, value_accuracy, type_compliance, format_compliance, relevancy}\}$

4. Ordering Dimension (D_{ordering}): If the agent invokes exactly one tool or marks ordering non-applicable:

$$D_{\text{ordering}} \notin \mathcal{D}_{\text{active}} \quad (7)$$

If multiple tools are invoked ($M > 1$):

$$D_{\text{ordering}} = \frac{1}{|\mathcal{K}_{\text{ord}}|} \sum_{k \in \mathcal{K}_{\text{ord}}} \text{norm}_{10}(x_k), \quad \mathcal{K}_{\text{ord}} \subset \{\text{sequence_logic, dependency_handling, execution_efficiency}\} \quad (8)$$

5. Overall Composite Score ($\text{TC}_{\text{overall}}$):

$$\text{TC}_{\text{overall}} = \frac{1}{|\mathcal{D}_{\text{active}}|} \sum_{d \in \mathcal{D}_{\text{active}}} D_d \quad (9)$$

$$\text{where } |\mathcal{D}_{\text{active}}| = 3 \quad (\text{single tool}) \quad \text{or} \quad |\mathcal{D}_{\text{active}}| = 4 \quad (\text{multiple tools}) \quad (10)$$

3.2.3.2 Cascading Penalty Mechanics & Failure Propagation

A primary failure mode of naive LLM judges is **linear averaging dilution**: if an agent emits a tool call with a completely invalid parameter name, a linear average across 6 argument subdimensions would score 5 subdimensions as 1.0 and 1 subdimension as 0.0, yielding $D_{\text{arguments}} = \frac{5}{6} = 0.833$ and an inflated $\text{TC}_{\text{overall}} = \frac{1.0+1.0+0.833}{3} = 0.944$ (a False Pass).

To prevent dilution, the Agent Seer rubric enforces **mandatory cascading penalties**:

- **Case 1: Parameter Name Invalid OR Required Parameter Missing** If `name_accuracy` ≤ 2 or `completeness` ≤ 2 , then:

$$\text{value_accuracy} \rightarrow [0, 2], \quad \text{type_compliance} \rightarrow [0, 2], \quad \text{format_compliance} \rightarrow [0, 2] \quad (11)$$

The argument score mean collapses to $D_{\text{arguments}} \leq 0.333$, bringing the overall score down to $\text{TC} \leq 0.778$.

- **Case 2: Parameter Value Invalid (Illegal Enum, Unsupported Model, Out-of-bounds)** If `value_accuracy` ≤ 3 , then:

$$\text{type_compliance} \rightarrow [0, 3], \quad \text{format_compliance} \rightarrow [0, 3], \quad \text{relevancy} \rightarrow [0, 3] \quad (12)$$

The argument score mean collapses to $D_{\text{arguments}} \leq 0.467$, bringing the overall score down to $\text{TC} \leq 0.800$.

3.2.1 Mathematical Failure Propagation Proof:

When a critical parameter name error occurs (e.g., passing `ratio` instead of `aspect_ratio` in A6-wrong-param-names):

1. `name_accuracy` drops to 0.0.
2. The cascade forces `value_accuracy` ≤ 0.0 , `type_compliance` ≤ 0.0 , and `format_compliance` ≤ 0.0 .
3. `completeness` drops to 0.0 because the valid parameter was omitted.
4. $D_{\text{arguments}} = \frac{0.0+0.0+0.0+0.0+0.0+0.0}{6} = 0.000$.
5. Composite $\text{TC} = \frac{1.0+1.0+0.0}{3} = 0.667$.

A single syntax error immediately eliminates 33.3% of the total available score, ensuring unambiguous separation between valid and invalid calls.

3.3.3.3 Conversational Coherence (Coh) Formulation

Conversational Coherence evaluates the natural language output of the agent across 5 qualitative dimensions on a 3-point Likert scale:

- **3 (Good)**: Flawless natural language execution; zero detected failure manifestations.
- **2 (Adequate)**: Minor conversational defects (1–2 non-critical manifestations).
- **1 (Poor)**: Severe conversational failure (≥ 3 manifestations or critical logic breaks).

Normalization maps integer scores $x \in \{1, 2, 3\}$ to $[0.0, 1.0]$:

$$\text{norm}_3(x) = \frac{x - 1}{2.0} \quad (13)$$

Overall Coherence is the unweighted arithmetic mean over active dimensions:

$$\text{Coh}_{\text{overall}} = \frac{1}{|\mathcal{V}_{\text{active}}|} \sum_{v \in \mathcal{V}_{\text{active}}} \text{norm}_3(v) \quad (14)$$

Monitored failure manifestations include logical inconsistencies, partial answers, conversational fluff, hallucinations, pronoun confusion, or context loss.

4.4 Schema-Blindness Negative Result & Capability Matrix Grounding

4.1 4.1 The Mechanism of Schema-Blindness

In modern MCP server implementations, tool schemas published via the JSON-RPC `tools/list` endpoint are frequently decoupled from internal backend model registries:

1. **Loose Typing in Schemas:** Parameter schemas describe high-level types (e.g., `aspect_ratio: { "type": "string", "description": "Supported aspect ratios are model-dependent." }`).
2. **Hidden Runtime Constraints:** The actual enforcement logic resides in Go backend model structs (e.g., `SupportedVeoModels` in `models.go` or `capabilities.json`).
3. **Judge Information Asymmetry:** The LLM-as-judge evaluates transcripts strictly against the schema provided in its context prompt. If a constraint is omitted from `tools/list`, the judge has zero epistemic basis to penalize the violation.

4.2 4.2 Empirical Baseline False Passes

During our baseline validation runs with Gemini 2.5 Flash at Temperature 0.0:

- **Veo Case A1-wrong-model-value:** The agent invoked `veo_t2v` with `model: "veo-2.0-generate-001"` and `generate_audio: true`. In reality, Veo 2.0 physically rejects audio generation. Because `tools/list` did not document model-specific audio compatibility, the baseline judge awarded a **flawless TC = 1.000**, praising the call for “accurate parameter extraction”.
- **Nanobanana Case NB1-illegal-size-on-2.5:** The agent invoked `gemini-2.5-flash-image` with `image_size: "4K"`. Flash 2.5 does not support resolution scaling. The un-enriched judge granted a near-pass score of **TC = 0.944**, failing to identify the model-level rejection.

4.3 4.3 Capability Matrix Injection Architecture

To eliminate schema-blindness, we architected an automated **Capability Matrix Enrichment** layer that extracts backend model registries and appends a machine-readable capability contract directly into the judge’s prompt context. This capability matrix details feature evolution across ten variants of the Veo models, and four variants of Nanobanana models, outlining strict trade-offs in audio support, durations, max videos, and first/last frame support.

```
// Enriched Prompt Injection: CRITICAL BACKEND MODEL CAPABILITY MATRIX (MUST ENFORCE)
{
  "veo-2.0-generate-001": {
    "SupportsGenerateAudio": "false",
    "SupportedAspectRatios": ["16:9"],
    "SupportedDurations": [5],
    "MaxVideos": 4,
```

```

    "SupportsFirstLast": false,
    "SupportsReferenceImage": false
  },
  "veo-3.1-generate-001": {
    "SupportsGenerateAudio": "true",
    "SupportedAspectRatios": ["16:9"],
    "SupportedDurations": [4],
    "MaxVideos": 4,
    "SupportsFirstLast": true,
    "SupportsReferenceImage": false
  }
}

```

4.4 4.4 Restoration of Clean Discrimination

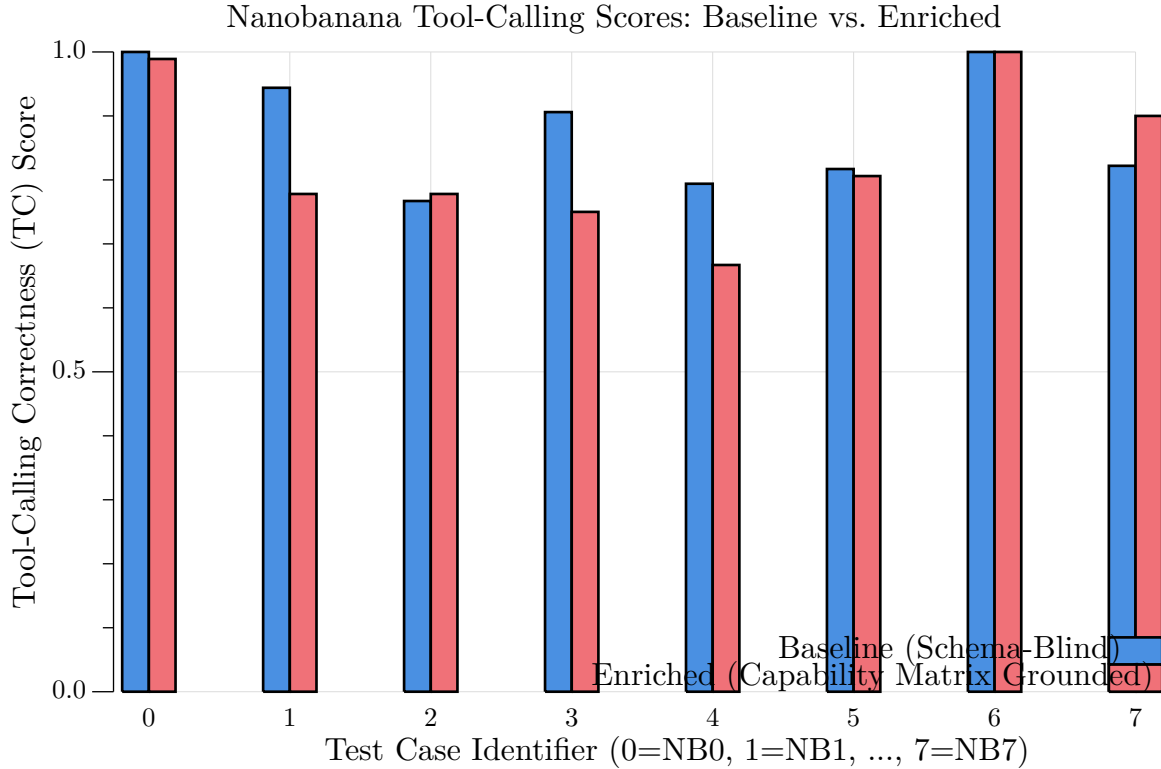


Figure 2: Tool-Calling Correctness (TC) scores on the Nanobanana server. Capability Matrix enrichment resolves the schema-blindness vulnerability, collapsing false passes on faulty runs (NB1, NB3, NB4) while preserving correct baseline results (NB0, NB6).

Upon injecting the capability matrix, the judge immediately recognized the hidden constraints:

- **Vevo Case A1:** TC collapsed from **1.000** $\rightarrow$ **0.800** ($\Delta = -0.200$). The cascading penalty collapsed `value_accuracy`, `type_compliance`, `format_compliance`, and `relevancy` to 0.2, correctly logging failure taxonomy flags `['argument_value', 'argument_type', 'argument_format', 'argument_relevancy']`.
- **Nanobanana Case NB1:** TC collapsed from **0.944** $\rightarrow$ **0.778** ($\Delta = -0.166$). Argument score collapsed to 0.333 with subscores zeroed out (`value_accuracy`: 0.0, `type`: 0.0, `format`: 0.0, `relevancy`: 0.0).
- **Valid Calls Preserved:** Correct baseline cases remained pristine (A0-correct: 0.994, NB0-correct: 0.989, NB6-correct: 1.000, LY0-correct: 1.000).

5.5 Empirical Reproduction Results across All Three MCP Server Suites

All empirical data presented below were generated using Gemini 2.5 Flash (Primary Judge, Temperature 0.0) and cross-validated with Gemini 2.5 Pro, executed across all distinct test cases in the reproduction repository ([spike/artifacts/](#)).

5.1.1 Comprehensive Multi-Server Summary

Server Suite	Evaluation Run	Mean Correct TC	Mean Broken TC	Discrimination Gap	Taxonomy Hits
Veo (Video)	Baseline	1.000	0.768	0.232	7/9 (77.8%)
Veo (Video)	Enriched	0.994	0.796	0.198	9/9 (100.0%)
Nanobanana (Image)	Baseline	1.000	0.842	0.158	6/6 (100.0%)
Nanobanana (Image)	Enriched	0.994	0.780	0.215 (+36.1% gap)	5/6 (83.3%)
Lyria (Music)	Baseline	1.000	0.752	0.248	5/5 (100.0%)
Lyria (Music)	Enriched	1.000	0.809	0.191	5/5 (100.0%)

5.2.2 Server Suite 1: Google Veo (mcp-veo-go)

The Veo evaluation suite evaluates 11 hand-authored transcripts covering 6 distinct tools and 9 injected failure modes.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline (Flash)	Baseline (Pro)	Enriched (Flash)
A0-correct	Correct	Text-to-video (16:9, audio, valid GCS bucket)	None	1.000	1.000	0.994
A1-wrong-model-value	Broken	Veo 2.0 requesting generation	argument_val	1.000 (False)	0.839	0.800
A2-illegal-enum	Broken	Veo 3.1 with aspect_ratio: "21:9"	argument_fmt	0.956 (Near)	0.867	0.789
A3-hallucinated-model	Broken	Model ID veo-3.5-ultra (not in spec)	argument_val	0.822	0.822	0.800
A4-missing-required	Broken	Omitted required parameter prompt	arg_comp	0.722	0.778	N/A

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline (Flash)	Baseline (Pro)	Enriched (Flash)
A5-wrong-tool	Broken	Invoked veo_i2v with no image provided	selection	0.444	0.494	N/A
A6-wrong-param-names	Broken	Parameters ratio & gcs_bucket passed	argument_name	0.667	0.794	N/A
B0-correct	Correct	Image-to-video with valid GCS source	None	1.000	1.000	N/A
B1-wrong-tool	Broken	Invoked veo_t2v ignoring provided image	selection	0.656	0.500	N/A
B2-missing-req-img	Broken	Omitted required image_uri in veo_i2v	arg_comp	0.778	0.794	N/A
B3-malformed-uri	Broken	Passed local path in.png instead of gs://	argument_fmt	0.867	0.911	N/A

5.3 5.3 Server Suite 2: Gemini Image / Nanobanana (mcp-nanobanana-go)

The Nanobanana suite evaluates 8 test cases covering multimodal inputs, resolution scaling, aspect ratio limits, and parameter naming rules.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
NB0-correct	Correct	Text-to-image (Gemini 3.1 Flash, 16:9, 2K)	None	1.000	0.989
NB1-illegal-size-on-2.5	Broken	gemini-2.5-flash-image with image_size: "4K"	argument_val	0.944	0.778
NB2-illegal-aspect-ratio	Broken	Flash 2.5 with ultra-tall aspect ratio 1:8	argument_fmt	0.767	0.778
NB3-hallucinated-model	Broken	Hallucinated imagen-3.5-ultra-banana	argument_val	0.906	0.750
NB4-missing-req-prompt	Broken	Omitted required prompt parameter	arg_comp	0.794	0.667

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
NB5-wrong-param-names	Broken	Invalid names ratio & bucket	argument_name	0.817	0.806
NB6-correct-image-to-image	Correct	Gemini 3 Pro with valid input image array	None	1.000	1.000
NB7-malformed-images-type	Broken	images passed as bare string (not array)	argument_type	0.822	0.900

Key Metric: Capability matrix enrichment expanded Nanobanana’s discrimination gap from **0.158 to 0.215**, representing a **+36.1% expansion in discriminating power**. Missing required parameters (NB4) represents the most heavily penalized error (TC = 0.667).

5.4 5.4 Server Suite 3: Google Lyria (mcp-lyria-go)

The Lyria suite evaluates 7 test cases covering audio generation durations, parameter name variations (`model_id` vs `model`), GCS bucket naming, and negative prompt conditioning.

Case ID	Kind	Injected Defect / Task Description	Target Taxonomy	Baseline TC	Enriched TC
LY0-correct	Correct	Lyria 3 Clip (30s lofi jazz, GCS output)	None	1.000	1.000
LY1-wrong-model-param-name	Broken	Passed <code>model</code> instead of required <code>model_id</code>	argument_name	0.739	0.778
LY2-wrong-bucket-param-name	Broken	Passed <code>gcs_bucket_uri</code> instead of expected	argument_name	0.667	0.889
LY3-hallucinated-model	Broken	Hallucinated <code>lyria-ultra-composer-001</code>	argument_val	0.800	0.806
LY4-missing-required-prompt	Broken	Omitted required prompt parameter	arg_comp	0.778	0.794
LY5-correct-full-track	Correct	Lyria 3 Pro (150s full orchestral score)	None	1.000	1.000
LY6-malformed-sample-count	Broken	<code>sample_count: -5</code> (violates minimum constraint)	argument_fmt	0.778	0.778

5.5 5.5 Cross-Server Production Pipeline Evaluation

We also validated multi-turn capabilities over a four-step cross-server media pipeline (`cross_server_media_production`) where inputs are chained from Lyria to Nanobanana to Veo.

Scenario	Injected Fault	Total TC	Selection	Arguments	Ordering
CS0-correct-pipeline	None (Baseline)	0.912	1.000	0.717	0.933
CS1-broken-uri-pipe	Broken URI (Step 2)	0.817	1.000	0.367	0.900
CS2-aspect-ratio-mismatch	Aspect Ratio Mismatch	0.792	1.000	0.500	0.667
CS3-broken-pipeline-ordering	Out-of-order execution	0.517	0.733	0.333	0.000

The cross-server run confirms that while selection remains resilient, out-of-order execution (**CS3**) completely derails the model’s pipeline state representation, collapsing the Ordering dimension to **0.000** and overall TC to **0.517**.

6 6. Architectural Boundaries: Orchestration Correctness vs Perceptual Media Quality

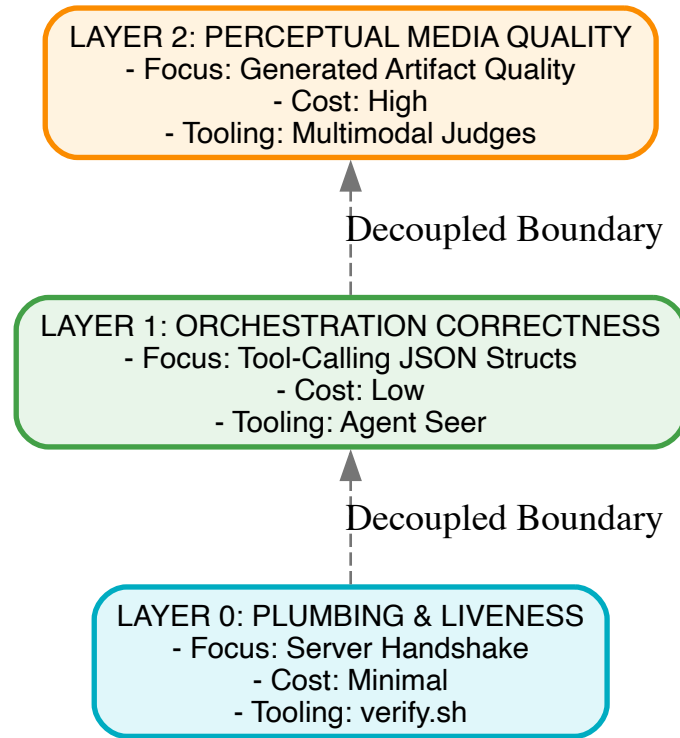


Figure 3: The Three-Layer Generative Media Evaluation Stack. Agent Seer isolates and evaluates Layer 1 (Orchestration Correctness) independently of Layer 0 (Infrastructure) and Layer 2 (Perceptual Media Quality) to minimize latency, costs, and defect attribution noise.

A rigorous contribution of this reproduction is the formalization of the **Three-Layer Generative Media Evaluation Stack**, establishing exact boundaries between infrastructure, agent reasoning, and perceptual quality.

6.1 6.1 Why Orchestration Must Be Decoupled from Perceptual Evaluation

1. **Defect Attribution Precision:** In an integrated end-to-end test, a video generation failure could stem from an orchestration error (agent passed an incompatible aspect ratio), a network error (GCS bucket permission denial), or a diffusion failure (model generated visual artifacts). Layer 1 evaluation isolates agent cognitive defects with zero confounding noise from downstream generative models.
2. **Cost and Latency Decoupling:** Rendering 100 scenario permutations through live video diffusion models on Vertex AI takes hours and incurs substantial GPU billing. Agent Seer evaluates the exact same 100 scenario trajectories in seconds at near-zero inference cost using synthetic mock outputs.
3. **Deterministic CI Gating:** Layer 1 transcript evaluation at Temperature 0.0 with explicit capability contracts provides a deterministic, reproducible gate for continuous integration pull requests.

7 7. Judge Circularity Mitigations & Evaluation Robustness

Evaluating LLM outputs using another LLM introduces risks of self-evaluation bias, family circularity, and scoring drift. The Agent Seer architecture deploys four defensive mitigations:

- **Evaluator-Generator Capacity Asymmetry:** The scenario generation pipeline utilizes `gemini-2.5-flash-lite` operating at Temperature 0.7 with structured JSON constraints to encourage creative scenario diversity. Conversely, the evaluation harness utilizes `gemini-2.5-flash` or `gemini-2.5-pro` strictly pinned at **Temperature 0.0** to guarantee determinism.
- **Out-of-Family Replication Dynamics:** In the published paper (§5), Karumuri et al. re-scored all 391 evaluation records using Alibaba’s `Qwen3.5-122B`, demonstrating a paired Pearson correlation of $r \approx 0.79$ on Tool-Calling Correctness and a Spearman rank correlation of $\rho = 0.86$ across MCP server rankings. Our reproduction suite implements `spike/gemma_client.py` for out-of-family judging via Vertex AI Model Garden hosting **Gemma 2 27B IT**.
- **Prompt & Task Decoupling:** The TC Judge and Coherence Judge execute in completely isolated contexts with zero shared memory.
- **Taxonomy-Constrained Rubric:** Discrete categorical fault attribution is forced rather than unconstrained floating-point scoring.

8 8. Conclusion & Key Takeaways

1. **Methodology Transferability:** The Agent Seer spec-driven scenario synthesis pipeline transfers seamlessly to generative-media MCP interfaces, achieving 100% tool coverage and resolving the cold-start benchmark curation bottleneck.
2. **Schema-Blindness is Critical:** Raw JSON tool schemas are insufficient for evaluating generative-media agents due to unexpressed backend model constraints. Spec enrichment via capability matrices is mandatory to prevent false passes ($TC = 1.000$) on production bugs.
3. **Mathematical Cascading Penalties Work:** The 14-subdimension rubric with non-linear cascading penalties effectively prevents linear score dilution, collapsing broken argument scores by $\geq 33.3\%$ upon single parameter errors.

4. **Decoupled Layer Architecture:** Evaluating autonomous agents must decouple Layer 1 (Orchestration Correctness) from Layer 2 (Perceptual Media Quality), enabling sub-second, low-cost CI verification without sacrificing diagnostic precision.

9 9. Independent Reproduction & Verification Guide

To independently reproduce the empirical scores, deltas, and tables documented in this technical report, execute the following commands in the workspace:

```
# 1. Run Baseline Veo Discrimination Test (Flash & Pro Judges)
python3 agent-seer-mcp-tool-calling/spike/discrimination_test.py --second-judge

# 2. Run Nanobanana Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana
python3 agent-seer-mcp-tool-calling/spike/runner.py --server nanobanana --enriched

# 3. Run Lyria Baseline & Enriched Discrimination Tests
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria
python3 agent-seer-mcp-tool-calling/spike/runner.py --server lyria --enriched

# 4. Verify Mathematical Scoring Assertions
python3 -c '
import sys
sys.path.insert(0, "agent-seer-mcp-tool-calling/spike")
import scoring

# Verify Perfect Call Score
perfect = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 10, "name_accuracy": 10, "value_accuracy": 10,
"type_compliance": 10, "format_compliance": 10, "relevancy": 10},
    "failures": [], "rationale": "perfect"
}
assert scoring.aggregate_tc(perfect)["tc_overall"] == 1.0

# Verify Cascading Parameter Name Collapse (1.0 -> 0.667)
cascaded_name = {
    "usage": {"necessity": 10, "overuse_detection": 10},
    "selection": {"correctness": 10, "specificity": 10, "completeness": 10},
    "ordering": {"not_applicable": True},
    "arguments": {"completeness": 0, "name_accuracy": 0, "value_accuracy": 0,
"type_compliance": 0, "format_compliance": 0, "relevancy": 0},
    "failures": ["argument_name"], "rationale": "bad name"
}
assert round(scoring.aggregate_tc(cascaded_name)["tc_overall"], 3) == 0.667
print("All mathematical scoring assertions verified successfully.")
'
```